

webRagSys Source Code

Generated: 2026-09-20 20:27

```
$ generate_source_pdfs.py . -e .py -o py_source_export/webRagSys_source.pdf
```

Contents

[illegible]

config.py

webRagSys¥config.py

55 lines | 1,735 bytes

```
1  # -*- coding: utf-8 -*-
2  """config.py - webRagSys全体の設定値集約.
3
4  Why: 設定値を1か所に集め、環境変数で切替可能にするため。
5  What: DB接続、LangChain、RAG、LLM偽装フラグを定義する。
6  Assumption / Dependencies: python-dotenv、pydantic-settingsに依存。
7  I/O: 入力=.env/環境変数、出力=Settingsインスタンス。
8  Caution: APIキー等の機密情報を直書きしないこと。
9  Future Work: 環境別 (dev/prod) 設定ファイル分割。
10 Change Log: 2026-09-19 初版作成。
11 """
12
13 from functools import lru_cache
14
15 from pydantic_settings import BaseSettings
16
17
18 # 設定値ブロック (冒頭集約)
19 class Settings(BaseSettings):
20     """アプリケーション設定値。"""
21
22     # 起動・DB関連の設定値
23     app_name: str = "webRagSys"
24     database_url: str = "postgresql://user:password@db:5432/webraq"
25     test_database_url: str = "sqlite:///./test.db"
26
27     # RAG関連の設定値 (LangChain用)
28     chunk_size: int = 500 # 1チャンクの文字数
29     chunk_overlap: int = 50 # チャンク間の重なり文字数
30     top_k: int = 3 # 類似検索の取得件数
31
32     # LLM関連の設定値 (偽実装切替)
33     use_fake: bool = True # True=課金なしFake、False=外部LLM API
34     openai_api_key: str = ""
35     openai_model: str = "gpt-4o-mini"
36     openai_embedding_model: str = "text-embedding-3-small"
37
38     model_config = {"env_file": ".env", "extra": "ignore"}
39
40
41 @lru_cache
42 def get_settings() -> Settings:
43     """設定値シングルトンを返す。
44
45     Args:
46         なし。
47
48     Returns:
49         Settings: 設定値インスタンス。
50
51     Side Effects:
52         なし (.env読込のみ)。
53     """
54     return Settings()
55
```

`__init__.py`

webRagSys¥src¥__init__.py

1 lines | 0 bytes

1

__init__.py

webRagSys¥src¥api¥__init__.py

1 lines | 0 bytes

1

main.py

webRagSys¥src¥api¥main.py

52 lines | 1,356 bytes

```
1  # -*- coding: utf-8 -*-
2  """src/api/main.py - FastAPI エントリポイント.
3
4  Why: API全体の起動点を一元化するため。
5  What: app生成とルータ登録、起動時DB初期化を行う。
6  Assumption / Dependencies: FastAPI、routers。
7  I/O: 入力=HTTP、出力=JSON。
8  Caution: なし。
9  Future Work: 認証ミドルウェア追加。
10 Change Log: 2026-09-19 初版作成。
11 """
12
13 from fastapi import FastAPI
14
15 from src.api.routers import chat, documents
16 from src.db.database import init_db
17
18
19 def create_app() -> FastAPI:
20     """アプリを生成する。
21
22     Returns:
23         FastAPI: アプリインスタンス。
24
25     Side Effects:
26         ルータ登録、起動時DB初期化。
27     """
28     app = FastAPI(title="webRagSys")
29
30     @app.on_event("startup")
31     def startup() -> None:
32         """起動時処理。"""
33         try:
34             init_db()
35         except (OSError, RuntimeError) as exc:
36             # 解説: テスト時はDB未起動でもimportできるよう記録のみ行う。
37             import logging
38
39             logging.getLogger(__name__).warning("init_db skip: %s", exc)
40
41     @app.get("/health")
42     def health() -> dict[str, str]:
43         """疎通確認用。"""
44         return {"status": "ok"}
45
46     app.include_router(documents.router)
47     app.include_router(chat.router)
48     return app
49
50
51 app = create_app()
52
```

__init__.py

webRagSys\$src\$api\$routes\$__init__.py

1 lines | 0 bytes

1

chat.py

webRagSys\$src\$api\$routes\$chat.py

44 lines | 1,371 bytes

```
1  # -*- coding: utf-8 -*-
2  """src/api/routes/chat.py - AIチャットAPI.
3
4  Why: 質問に対してRAG回答を返すため。
5  What: POST /chatを提供する。
6  Assumption / Dependencies: FastAPI, RAG chain。
7  I/O: 入力=質問JSON、出力=回答JSON。
8  Caution: DB/LLM障害時は500系で返す。
9  Future Work: 会話履歴対応。
10 Change Log: 2026-09-19 初版作成。
11 """
12
13 from fastapi import APIRouter, Depends, HTTPException
14 from sqlalchemy.orm import Session
15
16 from src.db.database import get_db
17 from src.rag.chain import answer_question
18 from src.schemas.chat import ChatRequest, ChatResponse
19
20 router = APIRouter(tags=["chat"])
21
22
23 @router.post("/chat", response_model=ChatResponse)
24 def chat(payload: ChatRequest, db: Session = Depends(get_db)) -> ChatResponse:
25     """質問に回答する。
26
27     Args:
28         payload: 質問内容。
29         db: DBセッション。
30
31     Returns:
32         ChatResponse: 回答と参照ID。
33
34     Raises:
35         HTTPException: 内部障害時500。
36     """
37     try:
38         answer, source_ids = answer_question(db, payload.question)
39     except RuntimeError as exc:
40         raise HTTPException(status_code=500, detail=str(exc)) from exc
41     except Exception as exc:
42         raise HTTPException(status_code=500, detail="回答生成に失敗しました") from exc
43     return ChatResponse(answer=answer, source_ids=source_ids)
44
```

documents.py

webRagSys\$src\$api\$routes\$documents.py

121 lines | 3,773 bytes

```
1  # -*- coding: utf-8 -*-
2  """src/api/routes/documents.py - 文書CRUD API.
3
4  Why: 文書管理機能を提供するため。
5  What: 一覧・詳細・登録・更新・削除を提供する。
6  Assumption / Dependencies: FastAPI、SQLAlchemy。
7  I/O: 入力=JSON／パスID、出力=JSON。
8  Caution: 存在しないIDは404、不正入力は422を返す。
9  Future Work: ページング、カテゴリ絞込。
10 Change Log: 2026-09-19 初版作成。
11 """
12
13 from fastapi import APIRouter, Depends, HTTPException
14 from sqlalchemy import select
15 from sqlalchemy.orm import Session
16
17 from src.db.database import get_db
18 from src.db.models import Document
19 from src.rag import chunker, vectorstore
20 from src.schemas.document import DocumentCreate, DocumentOut, DocumentUpdate
21
22 router = APIRouter(prefix="/documents", tags=["documents"])
23
24
25 def validate_input(title: str, content: str) -> None:
26     """入力チェックを行う。
27
28     Args:
29         title: タイトル。
30         content: 本文。
31
32     Raises:
33         HTTPException: 空文字の場合422。
34     """
35     if not title.strip() or not content.strip():
36         raise HTTPException(status_code=422, detail="titleとcontentは必須です")
37
38
39 def execute_logic_register(db: Session, payload: DocumentCreate) -> Document:
40     """文書登録の本体処理。
41
42     Args:
43         db: DBセッション。
44         payload: 登録内容。
45
46     Returns:
47         Document: 登録行。
48
49     Side Effects:
50         documents挿入、chunks保存。
51     """
52     doc = Document(title=payload.title, content=payload.content)
53     db.add(doc)
54     db.commit()
55     db.refresh(doc)
56     # 解説: RAG検索用に登録直後にチャンク化する。
57     vectorstore.save_chunks(db, doc.id, chunker.split_text(doc.content))
58     return doc
59
60
61 @router.get("", response_model=list[DocumentOut])
62 def list_documents(db: Session = Depends(get_db)) -> list[Document]:
63     """文書一覧を返す。"""
64     return list(db.execute(select(Document).order_by(Document.id)).scalars().all())
65
66
67 @router.get("/{doc_id}", response_model=DocumentOut)
68 def get_document(doc_id: int, db: Session = Depends(get_db)) -> Document:
69     """文書詳細を返す。
70
71     Raises:
```

```

72         HTTPException: 存在しない場合404。
73     """
74     doc = db.get(Document, doc_id)
75     if doc is None:
76         raise HTTPException(status_code=404, detail="文書が見つかりません")
77     return doc
78
79
80 @router.post("", response_model=DocumentOut, status_code=201)
81 def create_document(payload: DocumentCreate, db: Session = Depends(get_db)) -> Document:
82     """文書を登録する。"""
83     validate_input(payload.title, payload.content)
84     return execute_logic_register(db, payload)
85
86
87 @router.put("/{doc_id}", response_model=DocumentOut)
88 def update_document(
89     doc_id: int, payload: DocumentUpdate, db: Session = Depends(get_db)
90 ) -> Document:
91     """文書を更新する。
92
93     Raises:
94         HTTPException: 存在しない場合404。
95     """
96     validate_input(payload.title, payload.content)
97     doc = db.get(Document, doc_id)
98     if doc is None:
99         raise HTTPException(status_code=404, detail="文書が見つかりません")
100     doc.title = payload.title
101     doc.content = payload.content
102     db.commit()
103     db.refresh(doc)
104     vectorstore.save_chunks(db, doc.id, chunker.split_text(doc.content))
105     return doc
106
107
108 @router.delete("/{doc_id}", status_code=204)
109 def delete_document(doc_id: int, db: Session = Depends(get_db)) -> None:
110     """文書を削除する。
111
112     Raises:
113         HTTPException: 存在しない場合404。
114     """
115     doc = db.get(Document, doc_id)
116     if doc is None:
117         raise HTTPException(status_code=404, detail="文書が見つかりません")
118     vectorstore.delete_chunks(db, doc.id)
119     db.delete(doc)
120     db.commit()
121

```

`__init__.py`

webRagSys¥src¥db¥__init__.py

1 lines | 0 bytes

1

database.py

webRagSys¥src¥db¥database.py

79 lines | 2,539 bytes

```
1  # -*- coding: utf-8 -*-
2  """src/db/database.py - DB接続管理.
3
4  Why: 接続生成を一元化し、APIから再利用するため。
5  What: engine、SessionLocal、get_db、init_dbを提供する。
6  Assumption / Dependencies: SQLAlchemy、pgvector (Postgres時のみ)。
7  I/O: 入力=DATABASE_URL、出力=Session。
8  Caution: APIキー等の直書き禁止。LIKE検索でSQL組立しないこと。
9  Future Work: コネクションプール調整。
10 Change Log: 2026-09-19 初版作成。
11 """
12
13 import os
14 from collections.abc import Generator
15
16 from sqlalchemy import text
17 from sqlalchemy.orm import Session, declarative_base, sessionmaker
18
19 from config import get_settings
20
21 settings = get_settings()
22
23 # 設定値ブロック (冒頭集約)
24 # 解説: 環境変数DATABASE_URL優先。本番はPostgres、ドライバ欠落時はSQLiteに退避する。
25 DATABASE_URL = os.getenv("DATABASE_URL", settings.database_url)
26
27
28 def _create_engine():
29     """環境に応じたengineを生成する。"""
30     from sqlalchemy import create_engine as _ce
31
32     url = DATABASE_URL
33     if url.startswith("postgres"):
34         try:
35             import psycopg2 # noqa: F401 解説: 存在確認のみ。
36
37             return _ce(url)
38         except ImportError:
39             # 解説: テスト環境等でPGドライバなしでも起動できるよう退避する。
40             url = "sqlite:///./app.db"
41     connect_args = {"check_same_thread": False} if url.startswith("sqlite") else {}
42     return _ce(url, connect_args=connect_args)
43
44
45 engine = _create_engine()
46 SessionLocal = sessionmaker(bind=engine, autoflush=False, autocommit=False)
47 Base = declarative_base()
48
49
50 def get_db() -> Generator[Session, None, None]:
51     """DBセッションを供給する。
52
53     Yields:
54         Session: DBセッション。
55
56     Side Effects:
57         セッションを開閉する。
58     """
59     db = SessionLocal()
60     try:
61         yield db
62     finally:
63         db.close()
64
65
66 def init_db() -> None:
67     """テーブル初期化とpgvector拡張有効化。
68
69     Side Effects:
70         テーブル作成、拡張作成 (Postgres時のみ)。
71     """
```

```
72 from src.db import models # noqa: F401 解説: テーブル登録のため遅延importする。
73
74 # 解説: URL文字列ではなく実engineの方言で判定する (SQLite退避時は拡張不要)。
75 if engine.dialect.name == "postgresql":
76     with engine.begin() as conn:
77         conn.execute(text("CREATE EXTENSION IF NOT EXISTS vector"))
78 Base.metadata.create_all(bind=engine)
79
```

models.py

webRagSys¥src¥db¥models.py

64 lines | 2,227 bytes

```
1  #-*- coding: utf-8 -*-
2  """src/db/models.py - 文書テーブル定義.
3
4  Why: 文書とRAG用チャンクの保存先を定義するため。
5  What: documents、document_chunksを提供する。
6  Assumption / Dependencies: SQLAlchemy、pgvector (Postgres時のみVector型)。
7  I/O: 入力=Python属性、出力=DB行。
8  Caution: embedding列はPostgresではVector、SQLiteではJSON代替とする。
9  Future Work: カテゴリ・会話履歴テーブル追加。
10 Change Log: 2026-09-19 初版作成。
11 """
12
13 from datetime import datetime
14
15 from sqlalchemy import JSON, DateTime, ForeignKey, Integer, String, Text
16 from sqlalchemy.orm import Mapped, mapped_column
17
18 from src.db.database import Base, engine
19
20 # 設定値ブロック: embedding次元 (Fake時は小次元で十分)
21 EMBEDDING_DIM = 8
22
23
24 def _embedding_column() -> object:
25     """環境に応じたembedding列型を返す。
26
27     Returns:
28         object: Vector型 (Postgres) またはJSON (SQLite等)。
29
30     Side Effects:
31         なし。
32     """
33     if engine.url.drivername.startswith("postgresql"):
34         from pgvector.sqlalchemy import Vector
35
36         return mapped_column(Vector(EMBEDDING_DIM))
37     return mapped_column(JSON, default=list)
38
39
40 class Document(Base):
41     """文書テーブル。"""
42
43     __tablename__ = "documents"
44
45     id: Mapped[int] = mapped_column(Integer, primary_key=True, autoincrement=True)
46     title: Mapped[str] = mapped_column(String(200), nullable=False)
47     content: Mapped[str] = mapped_column(Text, nullable=False)
48     created_at: Mapped[datetime] = mapped_column(DateTime, default=datetime.utcnow)
49     updated_at: Mapped[datetime] = mapped_column(
50         DateTime, default=datetime.utcnow, onupdate=datetime.utcnow
51     )
52
53
54 class DocumentChunk(Base):
55     """RAG用チャンクテーブル。"""
56
57     __tablename__ = "document_chunks"
58
59     id: Mapped[int] = mapped_column(Integer, primary_key=True, autoincrement=True)
60     document_id: Mapped[int] = mapped_column(ForeignKey("documents.id"), nullable=False)
61     chunk_index: Mapped[int] = mapped_column(Integer, nullable=False)
62     content: Mapped[str] = mapped_column(Text, nullable=False)
63     embedding: Mapped[list] = _embedding_column() # type: ignore[assignment]
64
```

__init__.py

webRagSys\$src¥llm¥__init__.py

1 lines | 0 bytes

1

fake_provider.py

webRagSys¥src¥llm¥fake_provider.py

30 lines | 881 bytes

```
1  # -*- coding: utf-8 -*-
2  """src/llm/fake_provider.py - 課金なし偽LLM.
3
4  Why: テストと学習時に外部課金を避けるため。
5  What: 固定形式の偽回答を返す。
6  Assumption / Dependencies: なし。
7  I/O: 入力=質問+コンテキスト、出力=偽回答文。
8  Caution: 本番回答として使わないこと。
9  Future Work: なし。
10 Change Log: 2026-09-19 初版作成。
11 """
12
13
14 def fake_answer(question: str, context: str) -> str:
15     """偽回答を生成する。
16
17     Args:
18         question: 質問文。
19         context: 検索コンテキスト。
20
21     Returns:
22         str: 偽回答文。
23
24     Side Effects:
25         なし。
26     """
27     # 解説: 外部通信せず、RAG配線の確認用に定型文を返す。
28     head = context[:200].replace("¥n", " ")
29     return f"[FAKE回答] 質問「{question}」に対する関連情報: 「{head}」"
```

provider.py

webRagSys¥src¥llm¥provider.py

45 lines | 1,541 bytes

```
1  # -*- coding: utf-8 -*-
2  """src/llm/provider.py - LLM呼出切替層.
3
4  Why: 偽実装と本番APIを1か所で切替えるため。
5  What: generate_answerを提供する。
6  Assumption / Dependencies: langchain-openaiは本番時のみ必要。
7  I/O: 入力=質問+コンテキスト、出力=回答文。
8  Caution: 既定はFake。APIキーなしで本番指定時は例外にする。
9  Future Work: モデル別プロバイダ追加。
10 Change Log: 2026-09-19 初版作成。
11 """
12
13 from config import get_settings
14 from src.llm.fake_provider import fake_answer
15
16 settings = get_settings()
17
18
19 def generate_answer(question: str, context: str) -> str:
20     """回答文を生成する。
21
22     Args:
23         question: 質問文。
24         context: 検索コンテキスト。
25
26     Returns:
27         str: 回答文。
28
29     Raises:
30         RuntimeError: 本番設定でAPIキー未設定の場合。
31
32     Side Effects:
33         本番時のみ外部API通信する。
34     """
35     if settings.use_fake or not settings.openai_api_key:
36         return fake_answer(question, context)
37     try:
38         from langchain_openai import ChatOpenAI
39     except ImportError as exc:
40         raise RuntimeError("langchain-openai未導入") from exc
41     # 解説: LangChainのChatモデルにプロンプトを渡して回答を得る。
42     llm = ChatOpenAI(model=settings.openai_model, api_key=settings.openai_api_key)
43     prompt = f"以下の社内文書を基に回答してください。¥n¥n{context}¥n¥n質問: {question}"
44     return str(llm.invoke(prompt).content)
45
```

__init__.py

webRagSys\$src\$rag\$__init__.py

1 lines | 0 bytes

1

chain.py

webRagSys¥src¥rag¥chain.py

38 lines | 1,243 bytes

```
1  # -*- coding: utf-8 -*-
2  """src/rag/chain.py - RAG回答チェーン。
3
4  Why: 検索→整形→LLMの流れを1関数にまとめるため。
5  What: answer_questionを提供する（LangChain Retriever相当の薄い層）。
6  Assumption / Dependencies: vectorstore、llm.providerに依存。
7  I/O: 入力=DB+質問、出力=回答+参照ID。
8  Caution: なし。
9  Future Work: LCEL Chainへの置換、出典スコア付与。
10 Change Log: 2026-09-19 初版作成。
11 """
12
13 from sqlalchemy.orm import Session
14
15 from src.llm.provider import generate_answer
16 from src.rag import vectorstore
17
18
19 def answer_question(db: Session, question: str) -> tuple[str, list[int]]:
20     """RAGで質問に回答する。
21
22     Args:
23         db: DBセッション。
24         question: 質問文。
25
26     Returns:
27         tuple[str, list[int]]: 回答文と参照文書ID一覧。
28
29     Side Effects:
30         読取のみ（保存しない）。
31     """
32     # 解説: validate→検索→整形→生成の順にする。保存は行わない。
33     hits = vectorstore.search(db, question)
34     context = "¥n".join(h.content for h in hits)
35     answer = generate_answer(question, context)
36     source_ids = sorted([h.document_id for h in hits])
37     return answer, source_ids
38
```

chunker.py

webRag\$ys\$src¥rag¥chunker.py

56 lines | 1,704 bytes

```
1  #-*- coding: utf-8 -*-
2  """src/rag/chunker.py - 文書分割処理.
3
4  Why: 長文を検索可能な単位に分割するため (RAGの第一段階)。
5  What: LangChainのTextSplitter優先、なければ単純分割にフォールバックする。
6  Assumption / Dependencies: langchain_text_splitters任意。
7  I/O: 入力=本文str、出力=チャンクlist[str]。
8  Caution: なし。
9  Future Work: PDF／Word対応。
10 Change Log: 2026-09-19 初版作成。
11 """
12
13 from config import get_settings
14
15 settings = get_settings()
16
17 # 設定値ブロック (冒頭集約)
18 CHUNK_SIZE = settings.chunk_size
19 CHUNK_OVERLAP = settings.chunk_overlap
20
21
22 def split_text(
23     content: str, chunk_size: int = CHUNK_SIZE, overlap: int = CHUNK_OVERLAP
24 ) -> list[str]:
25     """本文をチャンク分割する。
26
27     Args:
28         content: 文書本文。
29         chunk_size: 1チャンクの文字数。
30         overlap: 重なり文字数。
31
32     Returns:
33         list[str]: チャンク一覧。
34
35     Side Effects:
36         なし。
37
38     Examples:
39         >>> split_text("abcdef", chunk_size=4, overlap=1)
40         ['abcd', 'def']
41     """
42     # 解説: LangChainがあればRecursiveCharacterTextSplitterを使い、なければ単純分割する。
43     try:
44         from langchain_text_splitters import RecursiveCharacterTextSplitter
45
46         splitter = RecursiveCharacterTextSplitter(
47             chunk_size=chunk_size, chunk_overlap=overlap
48         )
49         return splitter.split_text(content)
50     except ImportError:
51         chunks: list[str] = []
52         step = max(chunk_size - overlap, 1)
53         for i in range(0, len(content), step):
54             chunks.append(content[i : i + chunk_size])
55         return chunks
56
```

embeddings.py

webRagSys¥src¥rag¥embeddings.py

77 lines | 2,297 bytes

```
1  #-*- coding: utf-8 -*-
2  """src/rag/embeddings.py - Embedding生成.
3
4  Why: 質問と文書を意味ベクトル化して類似検索するため。
5  What: Fake（既定）とOpenAIの切替を提供する。
6  Assumption / Dependencies: langchain-openaiは本番時のみ必要。
7  I/O: 入力=text一覧、出力=ベクトル一覧。
8  Caution: APIキーを直書きしない。既定は課金なしFake。
9  Future Work: 国産Embedding対応。
10 Change Log: 2026-09-19 初版作成。
11 """
12
13 from config import get_settings
14
15 settings = get_settings()
16
17 # 設定値ブロック（冒頭集約）
18 EMBEDDING_DIM = 8 # Fakeベクトル次元（models.pyと一致させる）
19
20
21 def _fake_vector(text: str, dim: int = EMBEDDING_DIM) -> list[float]:
22     """決定的な偽ベクトルを生成する。
23
24     Args:
25         text: 対象文。
26         dim: 次元数。
27
28     Returns:
29         list[float]: 0~1の偽ベクトル。
30
31     Side Effects:
32         なし。
33     """
34     # 解説: 文字コード合計を種にした簡易ハッシュで、課金なしに再現性を保つ。
35     base = sum(ord(c) for c in text) % 100
36     return [(base + i * 13) % 100] / 100.0 for i in range(dim)]
37
38
39 def embed_texts(texts: list[str]) -> list[list[float]]:
40     """複数文をベクトル化する。
41
42     Args:
43         texts: 対象文一覧。
44
45     Returns:
46         list[list[float]]: ベクトル一覧。
47
48     Raises:
49         RuntimeError: 本番設定でAPIキー未設定の場合。
50
51     Side Effects:
52         本番時のみ外部API通信する。
53     """
54     if settings.use_fake or not settings.openai_api_key:
55         return [_fake_vector(t) for t in texts]
56     try:
57         from langchain_openai import OpenAIEmbeddings
58     except ImportError as exc:
59         raise RuntimeError("langchain-openai未導入") from exc
60     # 解説: 本番のみ外部通信する。テスト時は上記Fake分岐のため呼ばれない。
61     model = OpenAIEmbeddings(
62         model=settings.openai_embedding_model, api_key=settings.openai_api_key
63     )
64     return model.embed_documents(texts)
65
66
67 def embed_query(text: str) -> list[float]:
68     """質問文をベクトル化する。
69
70     Args:
71         text: 質問文。
```

```
72
73     Returns:
74         list[float]: 質問ベクトル。
75     """
76     return embed_texts([text])[0]
77
```

vectorstore.py

webRagSys¥src¥rag¥vectorstore.py

100 lines | 3,021 bytes

```
1  # -*- coding: utf-8 -*-
2  """src/rag/vectorstore.py - チャンク保存と類似検索.
3
4  Why: RAGの保存・検索層を分離するため。
5  What: save_chunks、searchを提供する。
6  Assumption / Dependencies: SQLAlchemy、pgvector (Postgres時のみ)。
7  I/O: 入力=文書ID+チャンク+ベクトル、出力=類似チャンク。
8  Caution: SQLite時はPython側でコサイン計算する (件数が少ない前提)。
9  Future Work: pgvectorインデックス (HNSW) 追加。
10 Change Log: 2026-09-19 初版作成。
11 """
12
13 import math
14
15 from sqlalchemy import delete, select
16 from sqlalchemy.orm import Session
17
18 from config import get_settings
19 from src.db.models import DocumentChunk
20 from src.rag.embeddings import embed_query, embed_texts
21
22 settings = get_settings()
23
24 # 設定値ブロック (冒頭集約)
25 TOP_K = settings.top_k
26
27
28 def _cosine(a: list[float], b: list[float]) -> float:
29     """コサイン類似度を計算する。
30
31     Args:
32         a: ベクトルA。
33         b: ベクトルB。
34
35     Returns:
36         float: 類似度 (1に近いほど類似)。
37     """
38     dot = sum(x * y for x, y in zip(a, b))
39     na = math.sqrt(sum(x * x for x in a)) or 1.0
40     nb = math.sqrt(sum(y * y for y in b)) or 1.0
41     return dot / (na * nb)
42
43
44 def save_chunks(db: Session, document_id: int, chunks: list[str]) -> None:
45     """既存チャンクを置換して保存する。
46
47     Args:
48         db: DBセッション。
49         document_id: 文書ID。
50         chunks: チャンク一覧。
51
52     Side Effects:
53         document_chunks行を削除・挿入する。
54     """
55     # 解説: 更新・削除時に古いベクトルが残らないよう、置換方式にする。
56     db.execute(delete(DocumentChunk).where(DocumentChunk.document_id == document_id))
57     vectors = embed_texts(chunks)
58     for idx, (chunk, vec) in enumerate(zip(chunks, vectors)):
59         db.add(
60             DocumentChunk(
61                 document_id=document_id, chunk_index=idx, content=chunk, embedding=vec
62             )
63         )
64     db.commit()
65
66
67 def delete_chunks(db: Session, document_id: int) -> None:
68     """文書のチャンクを削除する。
69
70     Args:
71         db: DBセッション。
```

```

72         document_id: 文書ID.
73
74     Side Effects:
75         対象行を削除する。
76     """
77     db.execute(delete(DocumentChunk).where(DocumentChunk.document_id == document_id))
78     db.commit()
79
80
81 def search(db: Session, query: str, top_k: int = TOP_K) -> list[DocumentChunk]:
82     """質問に類似したチャンクを返す。
83
84     Args:
85         db: DBセッション。
86         query: 質問文。
87         top_k: 取得件数。
88
89     Returns:
90         list[DocumentChunk]: 類似度順チャンク。
91
92     Side Effects:
93         なし（読取のみ）。
94     """
95     qvec = embed_query(query)
96     rows = db.execute(select(DocumentChunk)).scalars().all()
97     scored = [(cosine(qvec, list(r.embedding or [])), r) for r in rows]
98     scored.sort(key=lambda x: x[0], reverse=True)
99     return [r for _, r in scored[:top_k]]
100

```

`__init__.py`

webRagSys¥src¥schemas¥__init__.py

1 lines | 0 bytes

1

chat.py

webRagSys¥src¥schemas¥chat.py

27 lines | 658 bytes

```
1  # -*- coding: utf-8 -*-
2  """src/schemas/chat.py - チャットAPIの入出力定義.
3
4  Why: 質問と回答の形式を固定するため。
5  What: ChatRequest、ChatResponseを提供する。
6  Assumption / Dependencies: Pydantic v2。
7  I/O: 入力=質問JSON、出力=回答JSON。
8  Caution: なし。
9  Future Work: 会話履歴ID、出典スコア追加。
10 Change Log: 2026-09-19 初版作成。
11 """
12
13 from pydantic import BaseModel, Field
14
15
16 class ChatRequest(BaseModel):
17     """質問リクエスト。"""
18
19     question: str = Field(min_length=1)
20
21
22 class ChatResponse(BaseModel):
23     """回答レスポンス。"""
24
25     answer: str
26     source_ids: list[int] = []
27
```

document.py

webRagSys¥src¥schemas¥document.py

42 lines | 1,020 bytes

```
1  # -*- coding: utf-8 -*-
2  """src/schemas/document.py - 文書APIの入出力定義.
3
4  Why: リクエスト検証とレスポンス整形を分離するため。
5  What: 作成・更新・参照スキーマを提供する。
6  Assumption / Dependencies: Pydantic v2。
7  I/O: 入力=JSON、出力=検証済みdict。
8  Caution: なし。
9  Future Work: カテゴリ追加。
10 Change Log: 2026-09-19 初版作成。
11 """
12
13 from datetime import datetime
14
15 from pydantic import BaseModel, Field
16
17
18 class DocumentCreate(BaseModel):
19     """文書登録リクエスト。"""
20
21     title: str = Field(min_length=1, max_length=200)
22     content: str = Field(min_length=1)
23
24
25 class DocumentUpdate(BaseModel):
26     """文書更新リクエスト。"""
27
28     title: str = Field(min_length=1, max_length=200)
29     content: str = Field(min_length=1)
30
31
32 class DocumentOut(BaseModel):
33     """文書レスポンス。"""
34
35     id: int
36     title: str
37     content: str
38     created_at: datetime
39     updated_at: datetime
40
41     model_config = {"from_attributes": True}
42
```

__init__.py

webRagSys\src\utils__init__.py

1 lines | 0 bytes

1

`__init__.py`

webRagSys¥tests¥__init__.py

1 lines | 0 bytes

1

conftest.py

webRagSys¥tests¥conftest.py

75 lines | 2,103 bytes

```
1  # -*- coding: utf-8 -*-
2  """tests/conftest.py - テスト用DB設定.
3
4  Why: 本番DBを汚さずSQLiteで検証するため。
5  What: テスト用engineとclientを提供する。
6  Assumption / Dependencies: pytest、fastapi、SQLAlchemy。
7  I/O: 入力=なし、出力=client。
8  Caution: なし。
9  Future Work: Postgres実DBテスト追加。
10 Change Log: 2026-09-19 初版作成。
11 """
12
13 import os
14 from pathlib import Path
15
16 os.environ["USE_FAKE"] = "true"
17
18 from collections.abc import Generator
19
20 import pytest
21 from fastapi.testclient import TestClient
22 from sqlalchemy import create_engine
23 from sqlalchemy.orm import Session, sessionmaker
24
25
26 @pytest.fixture()
27 def db(tmp_path: Path) -> Generator[Session, None, None]:
28     """テスト用DBセッションを返す（テストごとに独立ファイル）。
29
30     Args:
31         tmp_path: pytest一時ディレクトリ。
32
33     Yields:
34         Session: テスト用セッション。
35
36     Side Effects:
37         一時DBファイル作成。
38     """
39     from src.db import models # noqa: F401 解説: create_all前にテーブル登録する。
40     from src.db.database import Base
41
42     # 解説: 固定ファイル共有による間欠失敗を避けるため、テストごとに分離する。
43     test_engine = create_engine(
44         f"sqlite:///[{tmp_path}]/test.db", connect_args={"check_same_thread": False}
45     )
46     TestSession = sessionmaker(bind=test_engine, autoflush=False, autocommit=False)
47     Base.metadata.create_all(bind=test_engine)
48     session = TestSession()
49     try:
50         yield session
51     finally:
52         session.close()
53
54
55 @pytest.fixture()
56 def client(db: Session) -> Generator[TestClient, None, None]:
57     """APIテスト用クライアントを返す。
58
59     Args:
60         db: テスト用セッション。
61
62     Yields:
63         TestClient: テストクライアント。
64     """
65     from src.api.main import create_app
66     from src.db.database import get_db
67
68     app = create_app()
69
70     def override() -> Generator[Session, None, None]:
71         yield db
```

```
72
73     app.dependency_overrides[get_db] = override
74     return TestClient(app)
75
```

test_chat.py

webRagSys¥tests¥test_chat.py

33 lines | 1,197 bytes

```
1  #-*- coding: utf-8 -*-
2  """tests/test_chat.py - AIチャットテスト（偽実装）.
3
4  Why: 課金なしでRAG回答の配線を検証するため。
5  What: 質問送信、偽回答、参照ID、RAG検索を検証する。
6  Assumption / Dependencies: USE_FAKE=true、conftestのclient。
7  I/O: 入力=質問JSON、出力=回答JSON。
8  Caution: 外部通信しないこと。
9  Future Work: 本番LLMのモック追加。
10 Change Log: 2026-09-19 初版作成。
11 """
12
13 from fastapi.testclient import TestClient
14
15 from src.rag.chunker import split_text
16
17
18 def test_chat_uses_documents(client: TestClient) -> None:
19     """登録文書を利用した回答を検証する。"""
20     client.post("/documents", json={"title": "休暇規定", "content": "年次休暇は10日付与される"})
21     res = client.post("/chat", json={"question": "休暇は何日?"})
22     assert res.status_code == 200
23     body = res.json()
24     assert "FAKE回答" in body["answer"]
25     assert len(body["source_ids"]) >= 1
26
27
28 def test_chunker() -> None:
29     """分割処理を検証する。"""
30     chunks = split_text("abcdef", chunk_size=4, overlap=1)
31     assert chunks[0] == "abcd"
32     assert "".join(chunks).replace(" ", "") != ""
33
```

test_documents.py

webRagSys¥tests¥test_documents.py

56 lines | 2,151 bytes

```
1  # -*- coding: utf-8 -*-
2  """tests/test_documents.py - 文書CRUDテスト.
3
4  Why: 要件の文書管理5項目を検証するため。
5  What: 登録・取得・更新・削除・存在しないIDを検証する。
6  Assumption / Dependencies: pytest、conftestのclient。
7  I/O: 入力=API呼出、出力=検証結果。
8  Caution: なし。
9  Future Work: ページング追加時に拡張。
10 Change Log: 2026-09-19 初版作成。
11 """
12
13 from fastapi.testclient import TestClient
14
15
16 def test_create_and_get(client: TestClient) -> None:
17     """登録後に取得できることを検証する。"""
18     res = client.post("/documents", json={"title": "休暇規定", "content": "年次休暇は10日"})
19     assert res.status_code == 201
20     doc_id = res.json()["id"]
21     res = client.get(f"/documents/{doc_id}")
22     assert res.status_code == 200
23     assert res.json()["title"] == "休暇規定"
24
25
26 def test_list(client: TestClient) -> None:
27     """一覧取得を検証する。"""
28     client.post("/documents", json={"title": "A", "content": "a1"})
29     res = client.get("/documents")
30     assert res.status_code == 200
31     assert len(res.json()) >= 1
32
33
34 def test_update(client: TestClient) -> None:
35     """更新を検証する。"""
36     doc_id = client.post("/documents", json={"title": "旧", "content": "旧本文"}).json()["id"]
37     res = client.put(f"/documents/{doc_id}", json={"title": "新", "content": "新本文"})
38     assert res.status_code == 200
39     assert res.json()["title"] == "新"
40
41
42 def test_delete(client: TestClient) -> None:
43     """削除を検証する。"""
44     doc_id = client.post("/documents", json={"title": "消", "content": "消本文"}).json()["id"]
45     res = client.delete(f"/documents/{doc_id}")
46     assert res.status_code == 204
47     res = client.get(f"/documents/{doc_id}")
48     assert res.status_code == 404
49
50
51 def test_not_found(client: TestClient) -> None:
52     """存在しないIDで404を検証する。"""
53     assert client.get("/documents/99999").status_code == 404
54     assert client.put("/documents/99999", json={"title": "x", "content": "y"}).status_code == 404
55     assert client.delete("/documents/99999").status_code == 404
56
```

File Index

webRagSys¥config.py	3
webRagSys¥src¥__init__.py	4
webRagSys¥src¥api¥__init__.py	5
webRagSys¥src¥api¥main.py	6
webRagSys¥src¥api¥routers¥__init__.py	7
webRagSys¥src¥api¥routers¥chat.py	8
webRagSys¥src¥api¥routers¥documents.py	9-10
webRagSys¥src¥db¥__init__.py	11
webRagSys¥src¥db¥database.py	12-13
webRagSys¥src¥db¥models.py	14
webRagSys¥src¥llm¥__init__.py	15
webRagSys¥src¥llm¥fake_provider.py	16
webRagSys¥src¥llm¥provider.py	17
webRagSys¥src¥rag¥__init__.py	18
webRagSys¥src¥rag¥chain.py	19
webRagSys¥src¥rag¥chunker.py	20
webRagSys¥src¥rag¥embeddings.py	21-22
webRagSys¥src¥rag¥vectorstore.py	23-24
webRagSys¥src¥schemas¥__init__.py	25
webRagSys¥src¥schemas¥chat.py	26
webRagSys¥src¥schemas¥document.py	27
webRagSys¥src¥utils¥__init__.py	28
webRagSys¥tests¥__init__.py	29
webRagSys¥tests¥conftest.py	30-31
webRagSys¥tests¥test_chat.py	32
webRagSys¥tests¥test_documents.py	33